Data Exploration

This notebook provides an initial exploration of the Archelec corpus.
The goal is to better understand the structure of the dataset before applying more advanced NLP methods.

In particular, we focus on document length and vocabulary patterns, as these elements may influence both lexical and semantic representations.

```
from google.colab import files

uploaded = files.upload()
```

```
Scegli file legislatives (1).zip
• legislatives (1).zip(application/x-zip-compressed) - 7384519 bytes, last modified: 28/4/2026 - 100% done
Saving legislatives (1).zip to legislatives (1).zip
```

Dataset Access

The dataset must be uploaded manually when running the notebook (e.g., in Google Colab), as it is not stored locally within the repository.

This avoids dependency issues and ensures that the notebook remains portable across environments.

1.1 Extracting the dataset

The dataset is uploaded as a zip file and extracted inside the Colab environment.

```
import zipfile
import os

zip_path = "/content/legislatives (1).zip"  # change this if your uploaded file has a different name

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall("/content")

print("Extraction completed.")
```

```
Extraction completed.
```

```
os.listdir("/content")
```

```
['.config', 'legislatives (1).zip', 'text_files', 'sample_data']
```

1.2 Loading text files

Each manifesto is stored as a `.txt` file.
We load all text files into a pandas DataFrame with two columns:

- `id`: file name
- `text`: manifesto content

```
import pandas as pd

folder = "/content/text_files/1988/legislatives"

data = []

for filename in os.listdir(folder):
    if filename.endswith(".txt"):
        file_path = os.path.join(folder, filename)

        with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
            text = f.read()

        data.append({
            "id": filename,
            "text": text
        })

df = pd.DataFrame(data)

print("Number of documents:", len(df))
df.head()
```

```
Number of documents: 3628
```

	id	text
0	EL174_L_1988_06_003_01_1_PF_03.txt	Sciences Po / fonds CEVIPOF\nELECTIONS LÉGISLA...
1	EL175_L_1988_06_050_01_2_PF_01.txt	Michel ROCARD 1er Ministre\nvous demande d'app...
2	EL176_L_1988_06_056_06_1_PF_02.txt	Sciences Po / fonds CEVIPOF\nMajorité présiden...
3	EL175_L_1988_06_048_01_1_PF_03.txt	Sciences Po / fonds CEVIPOF\nREPUBLIQUE FRANCA...
4	EL177_L_1988_06_088_03_2_PF_01.txt	Elections législatives du 12 Juin 1988 DÉPARTE...

Passaggi successivi: Genera codice con df | New interactive sheet

1.3 Basic corpus statistics

We compute basic statistics to better understand the structure of the corpus.

Each document corresponds to a political manifesto from the 1988 French legislative elections.
The number of characters, words, and lines provides a first indication of how much information each document contains.

Given the OCR origin of the data, these statistics also help identify potential irregularities, such as unusually short texts or formatting issues.

```
df["num_characters"] = df["text"].apply(len)
df["num_words"] = df["text"].apply(lambda x: len(x.split()))
df["num_lines"] = df["text"].apply(lambda x: len(x.splitlines()))

df[["num_characters", "num_words", "num_lines"]].describe()
```

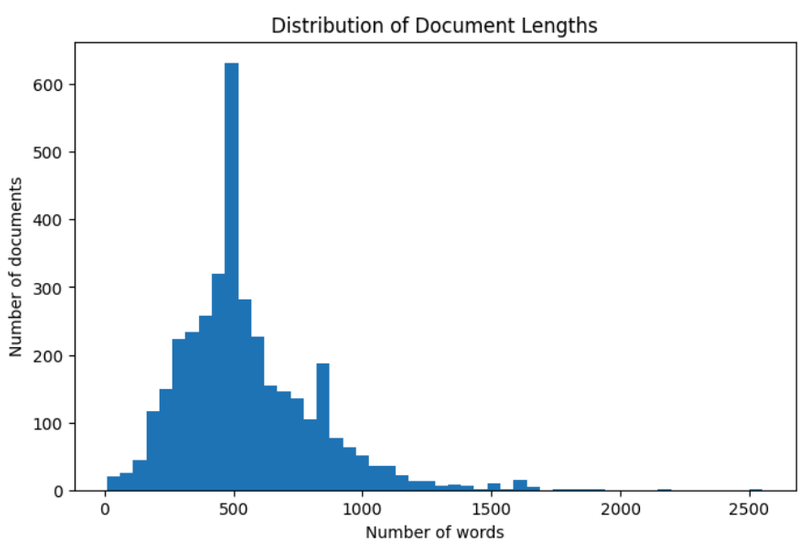
	num_characters	num_words	num_lines
count	3628.000000	3628.000000	3628.000000
mean	3446.140849	551.820011	32.327729
std	1632.790490	264.045134	12.680865
min	63.000000	9.000000	3.000000
25%	2404.000000	382.000000	25.000000
50%	3068.000000	495.000000	30.000000
75%	4236.250000	684.250000	37.000000
max	15489.000000	2552.000000	228.000000

1.4 Document length distribution

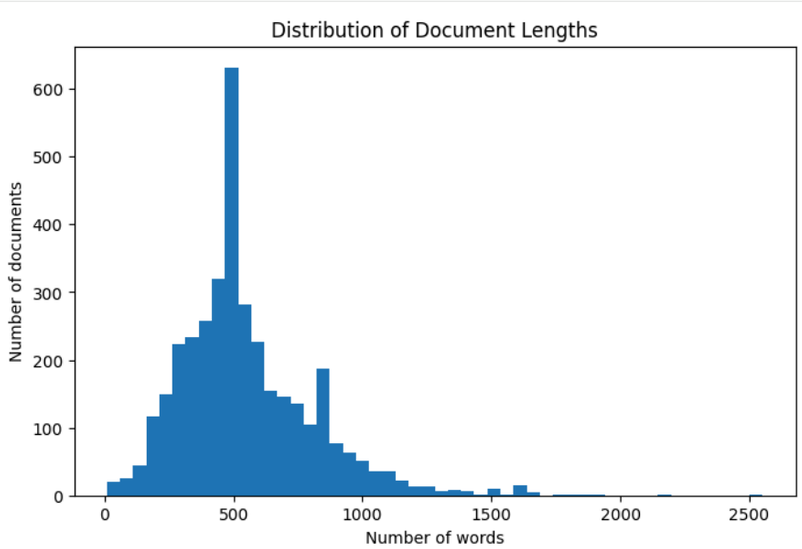
Document length is important because very short or very long documents may affect NLP representations and similarity scores.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(df["num_words"], bins=50)
plt.title("Distribution of Document Lengths")
plt.xlabel("Number of words")
plt.ylabel("Number of documents")
plt.show()
```




```
plt.figure(figsize=(8, 5))
plt.hist(df["num_words"], bins=50)
plt.title("Distribution of Document Lengths")
plt.xlabel("Number of words")
plt.ylabel("Number of documents")
plt.savefig("length_plot.png", dpi=300, bbox_inches="tight")
plt.show()
```



Document Length Distribution

The distribution of document lengths shows that the corpus is heterogeneous.
Most manifestos fall within a moderate range, but a smaller number of documents are significantly longer.

This suggests that candidates adopt different communication strategies.
Some texts are concise and focused, while others are more detailed and cover multiple topics.

These differences may affect downstream analysis, as longer documents typically include more vocabulary and more complex content.

1.5 Shortest and longest documents

We inspect the shortest and longest documents in the corpus.

Very short texts may indicate incomplete OCR extraction or missing content, while very long documents may correspond to more detailed political programs.

This step helps identify potential outliers and assess the overall quality of the dataset.

```
df.sort_values("num_words").head(10)[["id", "num_words", "num_characters"]]
```

	id	num_words	num_characters
2497	EL177_L_1988_06_075_08_1_BV_pdfmasterocr.txt	9	63
2951	EL176_L_1988_06_066_03_1_BV_pdfmasterocr.txt	16	92
748	EL177_L_1988_06_075_12_1_BV_pdfmasterocr.txt	18	137
946	EL177_L_1988_06_075_03_1_BV_pdfmasterocr.txt	18	133
2975	EL177_L_1988_06_075_18_1_BV_pdfmasterocr.txt	18	140
2029	EL177_L_1988_06_075_19_1_BV_pdfmasterocr.txt	18	134
1278	EL177_L_1988_06_075_13_1_BV_pdfmasterocr.txt	19	133
94	EL177_L_1988_06_075_21_1_BV_pdfmasterocr.txt	19	132
3623	EL177_L_1988_06_075_14_1_BV_pdfmasterocr.txt	21	153
863	EL176_L_1988_06_059_16_2_BV_pdfmasterocr.txt	25	180

```
df.sort_values("num_words", ascending=False).head(10)[["id", "num_words", "num_characters"]]
```

	id	num_words	num_characters
2484	EL174_L_1988_06_006_08_1_PF_02.txt	2552	15489
1803	EL175_L_1988_06_031_03_1_PF_03.txt	2160	13961
3579	EL177_L_1988_06_094_07_1_PF_03.txt	1939	11764
1655	EL177_L_1988_06_974_05_1_PF_03.txt	1853	12086
2327	EL175_L_1988_06_038_02_1_PF_pdfmasterocr.txt	1817	11571
2253	EL177_L_1988_06_094_07_1_PF_01.txt	1794	11413
465	EL176_L_1988_06_068_06_2_PF_01.txt	1782	11148
2696	EL176_L_1988_06_067_04_1_PF_01.txt	1663	10998
1281	EL174_L_1988_06_012_03_1_PF_03.txt	1653	9845
1699	EL174_L_1988_06_013_16_1_PF_05.txt	1648	10475

1.6 Average document length

We compute summary statistics such as mean and median document length.

The median is particularly useful in this context, as the distribution of document lengths may be skewed by a small number of very long manifestos.

These statistics provide a more robust view of what a "typical" document looks like in the corpus.

```
mean_words = df["num_words"].mean()
median_words = df["num_words"].median()
min_words = df["num_words"].min()
max_words = df["num_words"].max()

print("Mean number of words:", round(mean_words, 2))
print("Median number of words:", median_words)
print("Minimum number of words:", min_words)
print("Maximum number of words:", max_words)
```

```
Mean number of words: 551.82
Median number of words: 495.0
Minimum number of words: 9
Maximum number of words: 2552
```

1.7 Empty or very short documents

Very short documents may indicate OCR errors, incomplete files, or non-informative content.

Identifying these cases is important, as they may introduce noise in subsequent analyses, especially in similarity computations or embedding-based methods.

This step helps assess the reliability of the dataset.

```
short_docs = df[df["num_words"] < 50]

print("Number of very short documents:", len(short_docs))
short_docs[["id", "num_words", "text"]].head()
```

```
Number of very short documents: 16
```

	id	num_words	text
94	EL177_L_1988_06_075_21_1_BV_pdfmasterocr.txt	19	Sciences Po / fonds CEVIPOV\nDépartement : PAR...
610	EL176_L_1988_06_059_11_1_BV_pdfmasterocr.txt	34	RÉPUBLIQUE FRANÇAISE\nÉLECTIONS LÉGISLATIVES J...
636	EL176_L_1988_06_059_10_2_BV_pdfmasterocr.txt	37	Sciences Po / fonds CEVIPOV\n10e Circonscripti...
			Sciences Po / fonds CEVIPOV\nDépartement

1.8 Cleaning preview

Before applying NLP models, we define a simple text cleaning function.

The goal is not to fully normalize the text, but to reduce the most obvious OCR artifacts, such as irregular spacing, special characters, and formatting inconsistencies.

We compare raw and cleaned text to verify that the main linguistic content is preserved while noise is reduced.

```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r"\n+", " ", text)
    text = re.sub(r"[^a-zàâçéèêëîïôûùüÿñæœ\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["clean_text"] = df["text"].apply(clean_text)

df[["id", "text", "clean_text"]].head()
```

	id	text	clean_text
0	EL174_L_1988_06_003_01_1_PF_03.txt	Sciences Po / fonds CEVIPOF\nELECTIONS LÉGISLA...	sciences po fonds cevipof elections législativ...
1	EL175_L_1988_06_050_01_2_PF_01.txt	Michel ROCARD 1er Ministre\nvous demande d'app...	michel rocard er ministre vous demande d appor...
2	EL176_L_1988_06_056_06_1_PF_02.txt	Sciences Po / fonds CEVIPOF\nMajorité présiden...	sciences po fonds cevipof majorité présidentie...
		Sciences Po / fonds	sciences po fonds cevipof

1.9 Vocabulary size

We estimate the size of the vocabulary before and after cleaning.

This allows us to measure how much noise is removed by preprocessing.
A reduction in vocabulary size typically indicates that irrelevant tokens (e.g., OCR artifacts or special characters) have been filtered out.

At the same time, it is important to ensure that meaningful words are not excessively removed.

```
raw_vocab = set(" ".join(df["text"]).split())
clean_vocab = set(" ".join(df["clean_text"]).split())

print("Raw vocabulary size:", len(raw_vocab))
print("Cleaned vocabulary size:", len(clean_vocab))
```

```
Raw vocabulary size: 96753
Cleaned vocabulary size: 45429
```

1.10 Most frequent words

We examine the most frequent words in the cleaned corpus.

This provides a first overview of the language used in political manifestos.
Frequent terms often correspond to institutional, electoral, or rhetorical vocabulary.

This step helps identify recurring patterns, although it does not capture semantic meaning.

```
from collections import Counter

all_words = " ".join(df["clean_text"]).split()
word_counts = Counter(all_words)

word_counts.most_common(30)
```

```
[('de', 106904),
 ('la', 66712),
 ('et', 57951),
 ('le', 47506),
 ('l', 46671),
 ('les', 39795),
 ('à', 36985),
 ('pour', 35475),
 ('des', 32470),
 ('du', 27598),
 ('d', 27366),
 ('qui', 19085),
 ('en', 18147),
 ('une', 16534),
 ('vous', 15368),
```

```
('est', 15240),
('que', 14557),
('nous', 12263),
('un', 12210),
('dans', 11989),
('au', 11360),
('par', 10578),
('je', 10476),
('a', 9934),
('il', 9496),
('avec', 9013),
('notre', 8359),
('france', 8310),
('c', 8268),
('pas', 7809)]
```

Frequent Words Analysis

The most frequent words reflect the repetitive nature of political discourse.
Many of these terms are related to institutional or electoral contexts.

This indicates that political language relies on a relatively stable set of expressions.
However, frequency alone does not capture meaning, as similar words may be used in different contexts.

For this reason, lexical analysis will be complemented by semantic methods in the following notebooks.

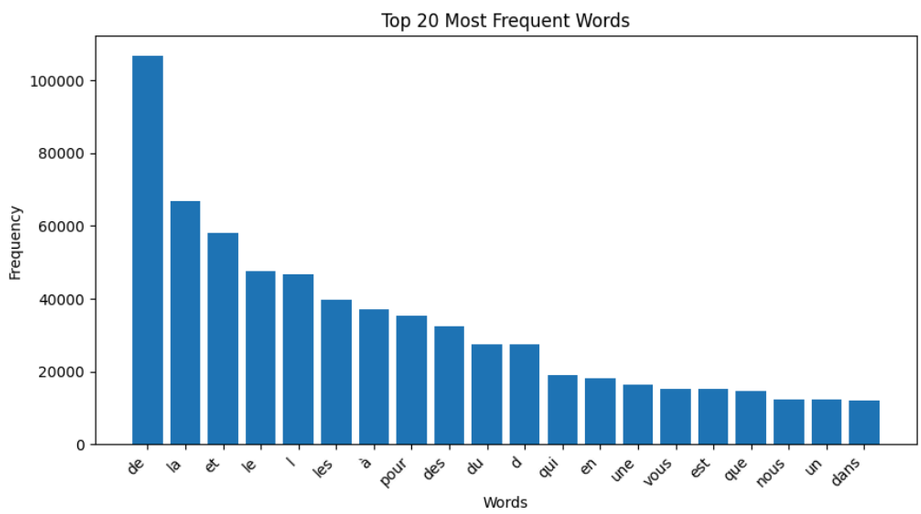
1.11 Word frequency visualization

We visualize the most frequent words in the corpus.

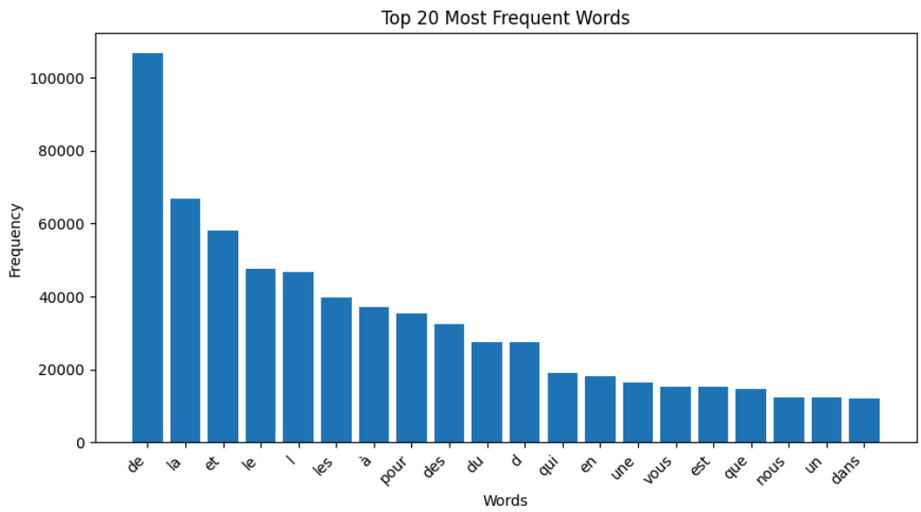
```
top_words = word_counts.most_common(20)

words = [w for w, c in top_words]
counts = [c for w, c in top_words]

plt.figure(figsize=(10, 5))
plt.bar(words, counts)
plt.title("Top 20 Most Frequent Words")
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.show()
```




```
plt.figure(figsize=(10, 5))
plt.bar(words, counts)
plt.title("Top 20 Most Frequent Words")
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.savefig("top_words_plot.png", dpi=300, bbox_inches="tight")
plt.show()
```



1.12 Summary

The exploratory analysis shows that the corpus contains several thousand political manifestos with heterogeneous characteristics.

Document length varies significantly, suggesting different communication strategies across candidates.
Some texts are concise, while others are more detailed and potentially cover multiple themes.

The presence of very short documents also highlights the impact of OCR noise, which may affect data quality.

Overall, this initial exploration confirms that the dataset is rich but not perfectly clean.
This motivates the use of preprocessing and more advanced NLP techniques in the following notebooks, in order to better capture both